Muhammad Adeel

23i-0739

CS-A

Department of Computer Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

Assignment#1

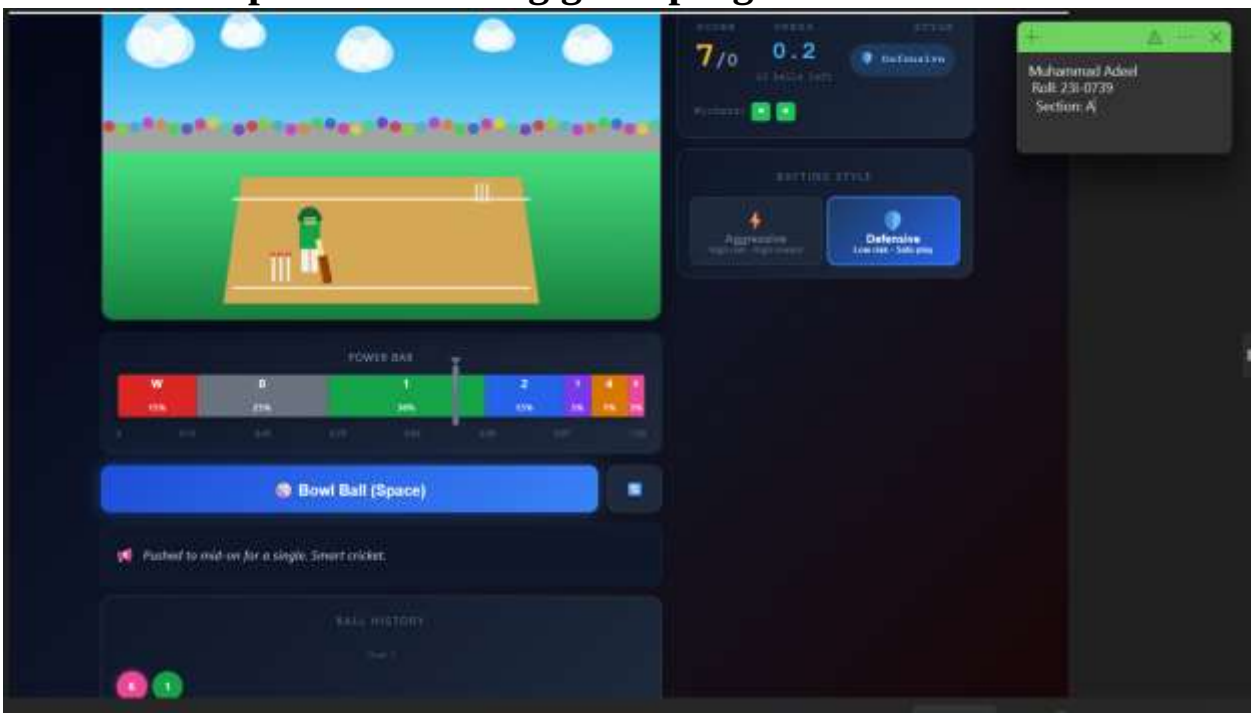
Report

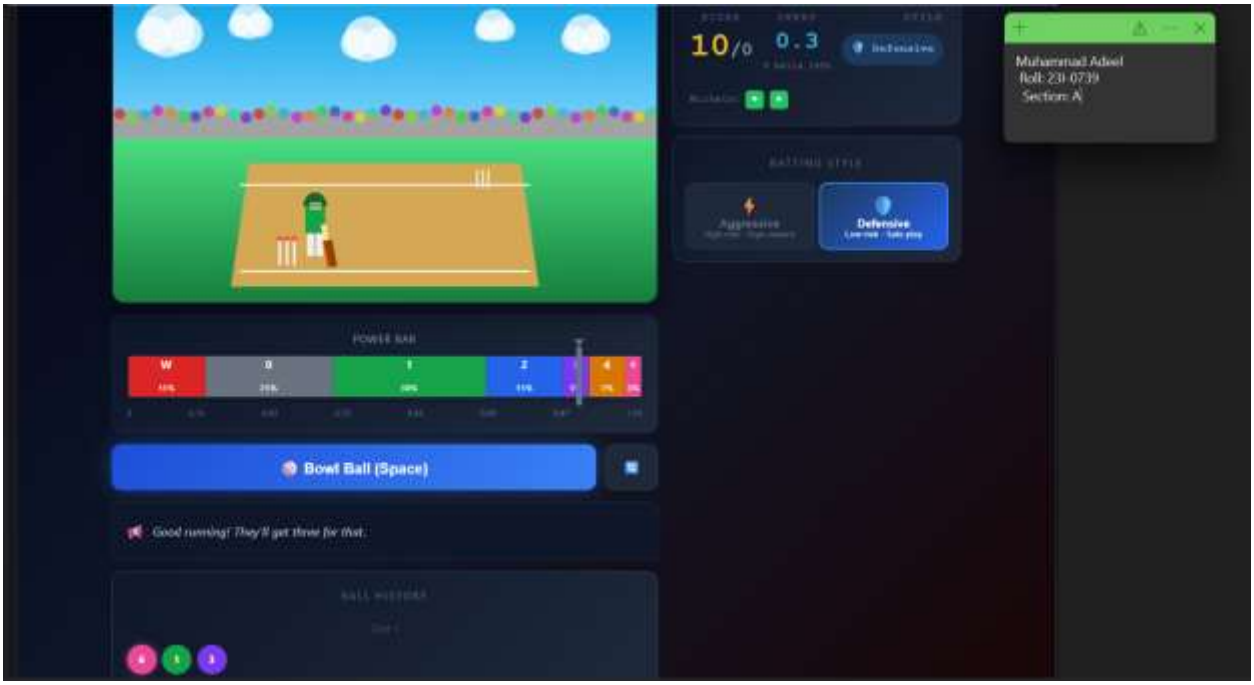
1. **Aggressive batting in action**

2. Defensive batting in action

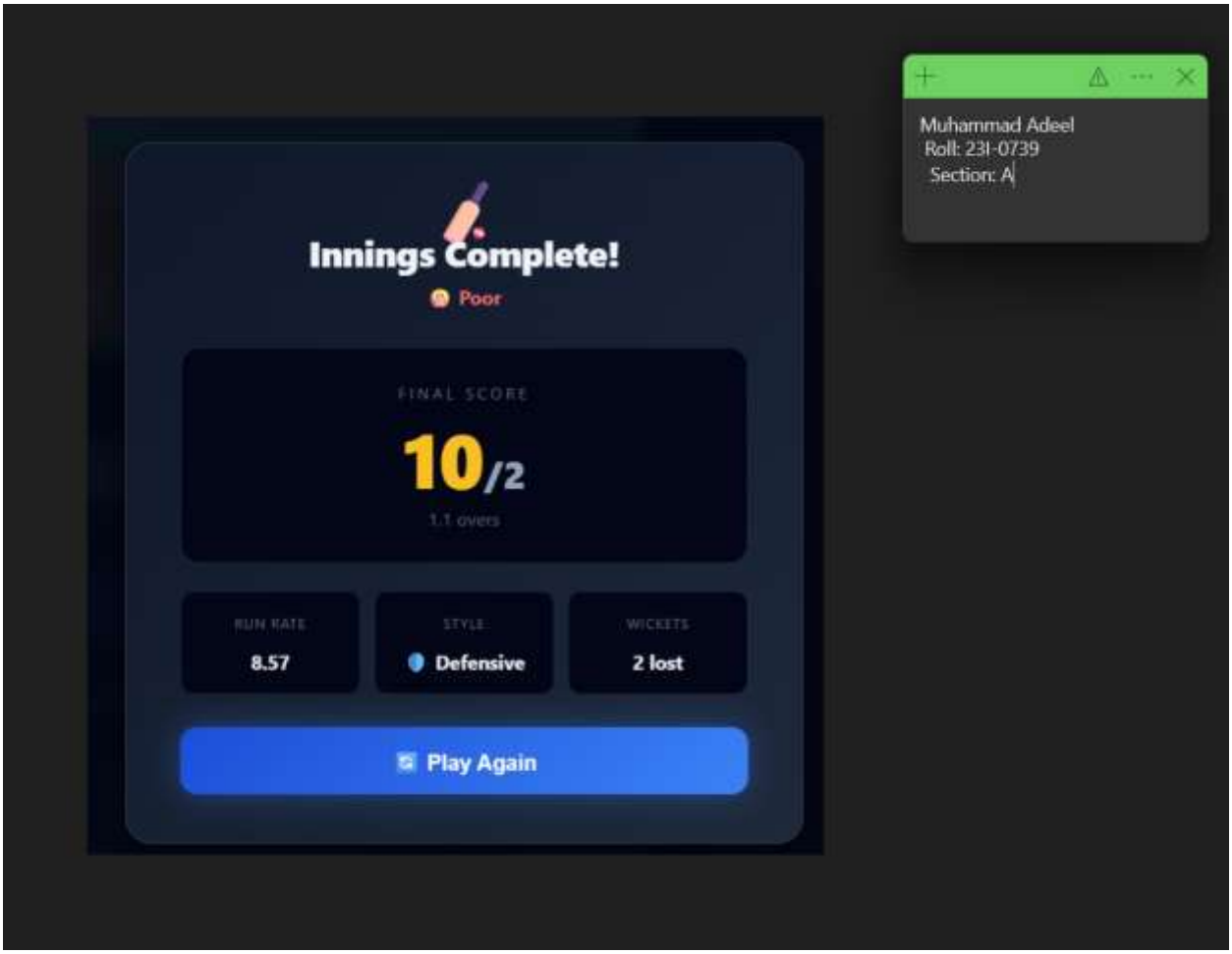
3. **Power bar visible during gameplay:**

4. **Scoreboard updates reflecting game progress:**





5. **Game over screen:**



Brief Description:

1. Game Logic and Working Mechanism

- The game is a 2D cricket batting simulator where the player bats for 12 balls (2 overs) with maximum 2 wickets allowed.
- The game follows a strict 5-phase state machine: **IDLE → BOWLING → BATTING → RESULT → IDLE (or GAMEOVER).**
- Player selects a batting style, clicks "Bowl Ball", then clicks "Play Shot" to freeze the power bar slider — the stopping position determines the outcome.
- Innings ends when all 12 balls are bowled or 2 wickets fall, showing a Game Over screen with final score and performance rating.

2. Mapping of Probabilities to the Power Bar

- Each batting style has 7 segments (Wicket, 0, 1, 2, 3, 4, 6) where every segment's probability value sums exactly to 1.0.
- Each segment occupies a width equal to **probability × 100%** on the power bar, so a 40% wicket zone takes 40% of the bar width.
- When the slider stops, the code loops through segments accumulating probabilities until the slider position falls within a segment's range — that segment's outcome is the result.
- Aggressive style has a 40% wicket zone (high risk) while Defensive style has only 15% (safer but fewer boundaries).

3. Implementation of Animations

- **Power Bar Slider** uses `requestAnimationFrame` at 60fps with a `useRef` to store position without causing re-renders, bouncing between 0 and 1.
- **Ball Animation** uses `performance.now()` to move the ball over 700ms with an ease-in-out curve making it start slow, speed up, then slow down.
- **Bat Swing** uses `setInterval` every 30ms to increment a `batPhase` value which the Canvas uses to rotate the bat arm smoothly.
- **Voice Commentary** uses the browser's built-in Web Speech API (`SpeechSynthesisUtterance`) to speak a random commentary line after every ball — no external library needed.